

Traducteur Multilingue

Français ↔ Anglais ↔ Ewe

Avec Reconnaissance et Synthèse Vocale

Rapport Technique Complet

Projet de fin de semestre — Master 1 IA-BD, EPL S2

Étudiants	ACOTIE Sam AKAKPO Koffi Samuel AMEDODZI Mawufe HOUNAKEY Etonam Josias MIDJIE Liman Romaric
Encadrante	Mme GBEDEVI
Filière	Master 1 Informatique — Intelligence Artificielle & Bases de Données
Institution	École Polytechnique de Lomé (EPL), Université de Lomé
Période	Mai – Juin 2026
Démo live	https://huggingface.co/spaces/josiasetnm/traducteur-multilingue-ewe

Rapport rédigé le 18 juin 2026

Table des matières

1	Introduction	4
1.1	Contexte et motivation	4
1.2	Objectifs du projet	4
1.3	Plan du rapport	5
2	Fondements théoriques	6
2.1	Qu'est-ce que l'intelligence artificielle ?	6
2.1.1	Machine learning (apprentissage automatique)	6
2.1.2	Deep learning (apprentissage profond) et réseaux de neurones	7
2.2	Traitement automatique du langage naturel (NLP)	7
2.2.1	Comment représenter les mots pour une machine ?	7
2.2.2	La tokenisation	7
2.3	L'architecture Transformer	8
2.3.1	Le mécanisme d'attention	8
2.3.2	Encodeur et décodeur	8
2.3.3	Le pré-entraînement et le fine-tuning	9
3	Collecte et prétraitement des données	10
3.1	Le problème des données en ewe	10
3.2	Sources de données utilisées	10
3.2.1	JW300 via OPUS	10
3.2.2	Codes de langue ISO	11
3.3	Prétraitement des données textuelles	11
3.3.1	Nettoyage	11
3.3.2	Filtrage	12
3.3.3	Division train / validation / test	12
4	Module de traduction automatique	13
4.1	Le modèle NLLB-200 de Meta	13
4.2	Pourquoi NLLB et pas Google Translate ?	13

4.3	Implémentation du service de traduction	13
4.4	Fonctionnement de la génération de texte	15
4.5	Les 6 paires de traduction supportées	15
5	Reconnaissance vocale (STT)	16
5.1	Qu'est-ce que la reconnaissance vocale ?	16
5.2	Whisper d'OpenAI	16
5.2.1	Comment Whisper traite l'audio	16
5.3	Le fine-tuning pour l'ewe : <code>crunch_modele_ewe</code>	17
5.3.1	Principe du fine-tuning audio	17
5.3.2	Transfer learning	17
5.4	Implémentation du service STT	17
5.5	Chargement du modèle : local ou Hugging Face Hub	19
6	Synthèse vocale (TTS)	20
6.1	Qu'est-ce que la synthèse vocale ?	20
6.2	Architecture TTS adoptée	20
6.2.1	gTTS pour le français et l'anglais	20
6.2.2	Coqui TTS VITS pour l'ewe	21
6.3	Implémentation du service TTS avec chargement paresseux	21
7	Architecture logicielle	23
7.1	Vue d'ensemble	23
7.2	Le pipeline multilingue	23
7.3	Gestion de la configuration	24
7.4	Structure complète du projet	24
8	Interface utilisateur	26
8.1	Choix de Streamlit	26
8.2	Organisation de l'interface	26
8.2.1	Onglet "Texte"	26
8.2.2	Onglet "Voix"	26
8.3	Optimisation : cache des modèles	27
9	Déploiement en ligne	28
9.1	Hugging Face Hub et Hugging Face Spaces	28
9.2	Hébergement du modèle STT	28
9.3	Containerisation avec Docker	29
9.4	Déploiement sur Hugging Face Spaces	30
10	Librairies utilisées	31

11 Évaluation et résultats	33
11.1 Métriques d'évaluation de la traduction	33
11.1.1 Score BLEU	33
11.1.2 Commande d'évaluation	33
11.2 Tests de l'interface	34
12 Limites et perspectives	35
12.1 Limites connues	35
12.1.1 Volume de données ewe	35
12.1.2 TTS ewe — corpus religieux	35
12.1.3 Performances en CPU	35
12.2 Perspectives d'amélioration	35
12.3 Conclusion	36
A Glossaire	37

Chapitre 1

Introduction

1.1 Contexte et motivation

La traduction automatique est l'une des applications les plus emblématiques de l'intelligence artificielle. Des systèmes comme Google Translate ou DeepL permettent aujourd'hui de traduire instantanément du texte entre des centaines de langues. Cependant, la grande majorité de ces systèmes se concentrent sur les langues dites à *hautes ressources* (l'anglais, le français, l'espagnol, etc.), c'est-à-dire les langues pour lesquelles il existe des volumes massifs de textes disponibles sur Internet ou dans des bases de données.

Les **langues africaines**, en revanche, sont presque toutes des langues à *faibles ressources* (*low-resource languages*). L'ewe (*Evegbe*), langue parlée par environ 7 millions de personnes au Togo et au Ghana, en est un exemple représentatif : elle dispose de très peu de données textuelles numériques, et pratiquement aucun outil de traduction automatique grand public n'en assure une couverture correcte.

Ce projet répond à ce manque en développant un **système de traduction multilingue** couvrant le triptyque **Français / Anglais / Ewe**, enrichi d'une **reconnaissance vocale** (l'utilisateur peut parler en ewe et obtenir la traduction en français) et d'une **synthèse vocale** (les traductions peuvent être lues à voix haute).

1.2 Objectifs du projet

1. Mettre en place un **pipeline de traduction automatique** pour les 6 paires de langues : fr $\leftrightarrow$ en, fr $\leftrightarrow$ ewe, en $\leftrightarrow$ ewe.
2. Intégrer un **modèle de reconnaissance vocale** (Speech-to-Text, STT) spécifiquement fine-tuné pour la langue ewe.

3. Intégrer une **synthèse vocale** (Text-to-Speech, TTS) pour le français, l'anglais et l'ewe.
4. Développer une **interface web** conviviale, déployée en ligne et accessible à n'importe quel utilisateur sans installation requise.

1.3 Plan du rapport

Ce rapport est organisé comme suit : après cette introduction, le chapitre 2 pose les fondements théoriques (IA, NLP, Transformers). Le chapitre 3 décrit la collecte et le prétraitement des données. Le chapitre 4 explique le module de traduction automatique. Le chapitre 5 détaille la reconnaissance vocale. Le chapitre 6 présente la synthèse vocale. Le chapitre 7 décrit l'architecture logicielle globale. Le chapitre 8 présente l'interface utilisateur Streamlit. Le chapitre 9 couvre le déploiement en ligne. Le chapitre 11 présente les résultats et l'évaluation. Enfin, le chapitre 12 conclut et trace des perspectives d'amélioration.

Chapitre 2

Fondements théoriques

2.1 Qu'est-ce que l'intelligence artificielle ?

Définition — Intelligence Artificielle

L'intelligence artificielle (IA) désigne l'ensemble des techniques permettant à une machine d'effectuer des tâches qui, si elles étaient réalisées par un être humain, nécessiteraient de l'intelligence : comprendre un texte, reconnaître une image, traduire une phrase, jouer aux échecs, etc.

L'IA repose sur plusieurs sous-domaines. Dans ce projet, nous mobilisons principalement le **machine learning** (apprentissage automatique) et, plus spécifiquement, le **deep learning** (apprentissage profond).

2.1.1 Machine learning (apprentissage automatique)

Le machine learning est une branche de l'IA dans laquelle on ne programme pas explicitement les règles que la machine doit suivre. Au lieu de cela, on lui fournit des **exemples** (données d'entraînement), et elle *apprend* elle-même les règles sous-jacentes à partir de ces exemples.

Analogie : Pour apprendre à un enfant ce qu'est un chat, on ne lui donne pas une définition formelle (“animal à 4 pattes, poils, moustaches...”). On lui montre des centaines de photos de chats et de non-chats, et il finit par reconnaître un chat même dans une situation qu'il n'a jamais vue. Le machine learning fonctionne sur le même principe.

2.1.2 Deep learning (apprentissage profond) et réseaux de neurones

Définition — Réseau de neurones artificiel

Un réseau de neurones artificiel est un modèle mathématique inspiré du cerveau humain. Il est composé de **neurones** (unités de calcul simples) organisés en **couches** successives. Chaque neurone reçoit des valeurs en entrée, les combine avec des **poids** (paramètres appris), et produit une valeur en sortie.

Un réseau “profond” (deep) possède de nombreuses couches intermédiaires (couches cachées). C’est cette profondeur qui lui permet de modéliser des relations très complexes dans les données.

2.2 Traitement automatique du langage naturel (NLP)

Définition — NLP (Natural Language Processing)

Le NLP est la branche de l’IA qui s’intéresse à la compréhension et à la génération du langage humain (texte ou parole) par les machines. Les tâches typiques de NLP incluent : la traduction automatique, la résumé de texte, la classification de sentiments, la réponse à des questions, la reconnaissance vocale, etc.

2.2.1 Comment représenter les mots pour une machine ?

Une machine ne comprend pas les mots directement — elle ne traite que des nombres. La première étape du NLP consiste donc à **transformer les mots en vecteurs numériques**, appelés *embeddings* (plongements de mots).

Principe : On attribue à chaque mot du vocabulaire un vecteur de nombres réels (par exemple un vecteur de 512 nombres). Les mots sémantiquement proches auront des vecteurs proches dans cet espace numérique. Par exemple, les vecteurs de “roi” et “reine” seront proches, et on peut même observer des relations algébriques comme :

$$\vec{roi} - \vec{homme} + \vec{femme} \approx \vec{reine}$$

2.2.2 La tokenisation

Avant de transformer les mots en vecteurs, il faut les **segmenter**. Ce processus s’appelle la **tokenisation**. Selon le modèle, un token peut être :

— un mot entier (“bonjour”),

- un sous-mot (“bon” + “jour”),
- un caractère.

Les modèles modernes (comme NLLB et Whisper) utilisent une tokenisation en sous-mots (*SentencePiece* ou *BPE* — *Byte Pair Encoding*), ce qui leur permet de gérer des mots rares ou inconnus en les décomposant.

2.3 L’architecture Transformer

Définition — Transformer

Un Transformer est une architecture de réseau de neurones introduite en 2017 par Vaswani et al. dans l’article “*Attention Is All You Need*”. Elle est aujourd’hui la base de presque tous les grands modèles de langage (GPT, BERT, Whisper, NLLB, etc.).

2.3.1 Le mécanisme d’attention

Le concept clé du Transformer est le **mécanisme d’attention** (*self-attention*). Il permet au modèle de *pondérer l’importance* de chaque mot d’une phrase par rapport aux autres mots, pour mieux comprendre le contexte.

Exemple : Dans la phrase “*La banque du fleuve est haute*”, le mot “banque” peut désigner un établissement financier ou un bord de fleuve. Le mécanisme d’attention permet au modèle de regarder le mot “fleuve” à proximité pour comprendre qu’il s’agit ici du sens géographique.

Mathématiquement, l’attention calcule pour chaque token une somme pondérée des valeurs des autres tokens :

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right) V$$

où Q (Query), K (Key) et V (Value) sont des projections linéaires des embeddings d’entrée, et d_k est la dimension des vecteurs clés.

2.3.2 Encodeur et décodeur

L’architecture Transformer originale comporte deux blocs principaux :

- **L’encodeur** lit la phrase source et la transforme en une représentation vectorielle riche en informations sémantiques.
- **Le décodeur** prend cette représentation et génère la phrase cible mot par mot, en s’appuyant sur les mots déjà générés et sur la sortie de l’encodeur (mécanisme

d'*attention croisée*).

Cette architecture encodeur-décodeur est utilisée pour toutes les tâches de *séquence à séquence* (seq2seq), comme la traduction automatique.

2.3.3 Le pré-entraînement et le fine-tuning

Pré-entraînement et Fine-tuning

Pré-entraîner un modèle signifie l'entraîner sur une très grande quantité de données génériques (des milliards de phrases), pour qu'il acquière une compréhension générale du langage. Ce processus demande d'immenses ressources de calcul (des semaines sur des centaines de GPU).

Fine-tuner (affiner) signifie reprendre ce modèle pré-entraîné et le ré-entraîner sur un jeu de données plus petit et plus spécifique à la tâche cible, pour l'adapter à nos besoins. Le fine-tuning est bien moins coûteux que le pré-entraînement.

Analogie : Imaginez un expert en langues qui parle couramment 100 langues (modèle pré-entraîné). Pour lui apprendre à traduire des textes médicaux spécifiques, il suffit de lui faire lire quelques milliers de documents médicaux (fine-tuning). Il n'a pas besoin de réapprendre les langues depuis zéro.

C'est exactement le principe que nous avons appliqué dans ce projet :

- Pour la **traduction** : nous utilisons NLLB-200, un modèle de Meta pré-entraîné sur 200 langues, qui supporte déjà l'ewe nativement.
- Pour la **reconnaissance vocale** : nous utilisons le modèle Whisper d'OpenAI, fine-tuné spécifiquement sur des données audio en ewe pour produire `crunch_modele_ewe`.

Chapitre 3

Collecte et prétraitement des données

3.1 Le problème des données en ewe

Toute solution de NLP repose fondamentalement sur des données. Pour entraîner ou fine-tuner un modèle, il faut des corpus de textes (ou d'audio) dans la langue cible. L'ewe est une langue à **très faibles ressources numériques** : contrairement au français ou à l'anglais, il n'existe pas de Wikipédia volumineux, pas de grands corpus web, ni de bases de données textuelles structurées facilement accessibles.

3.2 Sources de données utilisées

3.2.1 JW300 via OPUS

OPUS (<https://opus.nlpl.eu/>) est l'une des plus grandes collections de corpus multilingues libres. Le corpus JW300, issu des publications des Témoins de Jéhovah, est l'une des rares ressources disponibles pour l'ewe. Il contient environ 100 000 paires de phrases alignées entre l'ewe et le français/anglais.

La commande pour le télécharger via opustools :

```
1 from opustools import OpusRead
2
3 OpusRead(
4     directory='JW300',
5     source='fr',
6     target='ee',      # code ISO de l'ewe
7     release='v1',
```

```

8     preprocess='raw',
9     write=['jw300_fr_ewe.src', 'jw300_fr_ewe.tgt'],
10    download_dir='./data'
11 )

```

Listing 3.1 – Téléchargement du corpus JW300 fr-ewe

3.2.2 Codes de langue ISO

Chaque langue est identifiée par un code ISO 639-1 ou 639-2 :

Langue	Code ISO	Code NLLB
Français	fr	fra_Latn
Anglais	en	eng_Latn
Ewe	ee	ewe_Latn

3.3 Prétraitement des données textuelles

Le prétraitement est l'ensemble des opérations permettant de “nettoyer” et de standardiser les données brutes avant de les utiliser pour l'entraînement.

3.3.1 Nettoyage

```

1  import re
2  import unicodedata
3
4  def clean_text(text: str) -> str:
5      # Normalisation Unicode (important pour l'ewe qui a des
6      #   tons)
7      text = unicodedata.normalize('NFC', text)
8      # Suppression des caractères indésirables
9      text = re.sub(r'[\w\s\u0300-\u036f.,!?:;\'-]', '', text)
10     # Réduction des espaces multiples
11     text = re.sub(r'\s+', ' ', text).strip()
12     return text

```

Listing 3.2 – Nettoyage des textes

Pourquoi la normalisation Unicode est-elle cruciale pour l'ewe ? L'ewe utilise des caractères spéciaux comme le η (eta), le d (d rétroflexe) et le f (f à crochet). Sans normalisation, le même caractère peut être encodé de plusieurs façons différentes, ce qui crée des incohérences dans les données.

3.3.2 Filtrage

On supprime les paires de phrases trop courtes, trop longues, ou avec un ratio de longueur anormal entre la source et la cible :

```
1 df = df[df['fr_text'].str.split().str.len().between(2, 100)]  
2 df = df[df['ewe_text'].str.split().str.len().between(2, 100)]
```

Listing 3.3 – Filtrage des paires de phrases

3.3.3 Division train / validation / test

Les données sont divisées en trois ensembles :

- **Train (80%)** : données sur lesquelles le modèle apprend.
- **Validation / Dev (10%)** : données utilisées pendant l'entraînement pour surveiller les performances et éviter le surapprentissage.
- **Test (10%)** : données jamais vues, utilisées uniquement pour l'évaluation finale.

Chapitre 4

Module de traduction automatique

4.1 Le modèle NLLB-200 de Meta

Définition — NLLB-200

NLLB (*No Language Left Behind*) est un modèle de traduction automatique développé par Meta AI et publié en 2022. Il est capable de traduire entre **200 langues** différentes, dont l’ewe (`ewe_Latn`). C’est l’un des rares modèles grand public supportant cette langue.

Nous utilisons la variante `nllb-200-distilled-600M` : un modèle de 600 millions de paramètres, obtenu par *distillation* du modèle complet (3,3 milliards de paramètres). La distillation est une technique qui “compresse” un grand modèle en un modèle plus petit, plus rapide, tout en conservant la majorité de ses performances.

4.2 Pourquoi NLLB et pas Google Translate ?

- NLLB est **open source** et **gratuit**, utilisable sans quota ni clé API payante.
- Il peut être exécuté **localement** sur notre machine, sans dépendance Internet pendant l’inférence.
- Il peut être **fine-tuné** sur nos propres données ewe pour améliorer la qualité des traductions.
- Il supporte **toutes nos paires de langues** dans un seul modèle.

4.3 Implémentation du service de traduction

Le service de traduction est encapsulé dans la classe `TranslationService` :

```
1 from transformers import AutoModelForSeq2SeqLM, AutoTokenizer
```

```
2 import torch
3
4 LANG_CODE_MAP = {
5     "fr": "fra_Latn",
6     "en": "eng_Latn",
7     "ewe": "ewe_Latn",
8 }
9
10 class TranslationService:
11     def __init__(self, model_name: str, device: str = "cpu"):
12         self.tokenizer = AutoTokenizer.from_pretrained(
13             model_name)
14         self.model = AutoModelForSeq2SeqLM.from_pretrained(
15             model_name)
16         self.model.to(device)
17
18     def translate(self, text: str, source_lang: str,
19                 target_lang: str) -> str:
20         self.tokenizer.src_lang = LANG_CODE_MAP[source_lang]
21         inputs = self.tokenizer(text, return_tensors="pt")
22
23         with torch.no_grad():
24             generated = self.model.generate(
25                 **inputs,
26                 forced_bos_token_id=self.tokenizer.
27                     convert_tokens_to_ids(
28                         LANG_CODE_MAP[target_lang]
29                     ),
30                 max_new_tokens=128,
31             )
32         return self.tokenizer.batch_decode(
33             generated, skip_special_tokens=True
34         )[0]
```

Listing 4.1 – Service de traduction avec NLLB

Explication ligne par ligne :

- `AutoTokenizer.from_pretrained` : charge le tokenizer du modèle, qui sait comment découper le texte en tokens.
- `tokenizer.src_lang` : indique au tokenizer la langue source, ce qui influence la façon dont le texte est tokenisé.
- `forced_bos_token_id` : “force” le premier token généré à être le token de début

de la langue cible, ce qui oriente le modèle vers la traduction dans la bonne langue.

- `torch.no_grad()` : désactive le calcul des gradients pendant l'inférence, ce qui économise de la mémoire et accélère le traitement.
- `max_new_tokens=128` : limite la longueur maximale de la traduction générée.

4.4 Fonctionnement de la génération de texte

Le modèle génère la traduction **token par token**, de gauche à droite. À chaque étape, il calcule une distribution de probabilité sur tout le vocabulaire et sélectionne le token le plus probable (stratégie *greedy*) ou utilise des techniques plus sophistiquées comme le *beam search* (exploration de plusieurs hypothèses en parallèle).

$$P(w_t | w_1, w_2, \dots, w_{t-1}, \text{source}) = \text{softmax}(W \cdot h_t)$$

où h_t est l'état caché du décodeur à l'étape t , et W est une matrice de projection vers le vocabulaire.

4.5 Les 6 paires de traduction supportées

Source	Cible	Supporté
Français	Anglais	✓
Anglais	Français	✓
Français	Ewe	✓
Ewe	Français	✓
Anglais	Ewe	✓
Ewe	Anglais	✓

Toutes ces paires sont gérées par **un seul et même modèle** NLLB-200, via le paramètre `forced_bos_token_id` qui oriente la génération.

Chapitre 5

Reconnaissance vocale (STT)

5.1 Qu'est-ce que la reconnaissance vocale ?

Définition — STT (Speech-to-Text)

La reconnaissance vocale, ou *Speech-to-Text* (STT), est la tâche qui consiste à convertir un signal audio (la voix humaine) en texte écrit. C'est la même technologie utilisée par Siri, Google Assistant ou les sous-titres automatiques de YouTube.

5.2 Whisper d'OpenAI

Définition — Whisper

Whisper est un modèle de reconnaissance vocale développé par OpenAI et publié en 2022. Il est entraîné sur 680 000 heures d'audio multilingue et supporte 99 langues. Son architecture est également basée sur les Transformers, avec un encodeur audio et un décodeur de texte.

5.2.1 Comment Whisper traite l'audio

Le signal audio brut est d'abord converti en une représentation visuelle appelée **spectrogramme mel**. Un spectrogramme représente les fréquences sonores en fonction du temps — c'est une sorte d'“image” du son.

Audio $\xrightarrow{\text{STFT}}$ Spectrogramme $\xrightarrow{\text{Encodeur Transformer}}$ Représentation $\xrightarrow{\text{Décodeur Transformer}}$ Texte

Whisper rééchantillonne toujours l'audio à **16 000 Hz** (16 kHz) avant traitement. C'est pour cela que notre code utilise `librosa.load(..., sr=16000)`.

5.3 Le fine-tuning pour l'ewe : `crunch_modele_ewe`

Whisper, dans sa version originale, ne supporte pas l'ewe comme langue native. Pour combler ce manque, le modèle a été **fine-tuné** sur des données audio en ewe.

5.3.1 Principe du fine-tuning audio

Le fine-tuning d'un modèle STT consiste à lui fournir des paires :

$$(\text{fichier audio ewe}) \longleftrightarrow (\text{transcription texte ewe})$$

Le modèle apprend à ajuster ses paramètres pour minimiser l'erreur entre la transcription qu'il produit et la transcription correcte fournie. La métrique utilisée est le **WER** (*Word Error Rate* — taux d'erreur par mot) :

$$\text{WER} = \frac{S + D + I}{N} \times 100\%$$

où S = substitutions, D = suppressions, I = insertions, N = nombre total de mots dans la référence.

5.3.2 Transfer learning

Le transfer learning (apprentissage par transfert) est le fait de réutiliser les connaissances acquises par un modèle pré-entraîné sur une nouvelle tâche.

Dans notre cas :

1. Whisper a d'abord appris à reconnaître la parole dans 99 langues (pré-entraînement).
2. On reprend ses paramètres et on le ré-entraîne sur des données audio ewe (fine-tuning). Puisque Whisper comprend déjà la structure générale de la parole humaine, il n'a besoin que de peu d'exemples ewe pour s'y adapter.

C'est l'avantage majeur du transfer learning : on n'a pas besoin de repartir de zéro, ce qui réduit considérablement le volume de données et le temps d'entraînement nécessaires.

5.4 Implémentation du service STT

```
1 import librosa
2 import torch
```

```
3 from transformers import AutoModelForSpeechSeq2Seq,
   AutoProcessor
4 from pathlib import Path
5
6 LANG_PROMPT_MAP = {
7     "fr": "french",
8     "en": "english",
9     "ewe": None, # pas de contrainte de langue, le modele
   decide
10 }
11
12 class STTService:
13     def __init__(self, model_path: Path | str, device: str =
   "cpu"):
14         # Charge le modele depuis le disque local OU depuis
   Hugging Face Hub
15         self.processor = AutoProcessor.from_pretrained(
   model_path)
16         self.model = AutoModelForSpeechSeq2Seq.
   from_pretrained(model_path)
17         self.model.to(device)
18
19     def transcribe(self, audio_path: str,
20                   source_language: str = "ewe") -> str:
21         # 1. Charge l'audio et le reeechantillonne a 16 kHz
22         speech, sr = librosa.load(audio_path, sr=16000)
23
24         # 2. Convertit l'audio en features (spectrogramme mel
   )
25         inputs = self.processor(speech, sampling_rate=sr,
26                                return_tensors="pt")
27         features = inputs.input_features.to(self.device)
28
29         # 3. Configure la langue de transcription
30         whisper_lang = LANG_PROMPT_MAP.get(source_language)
31         if whisper_lang:
32             forced_ids = self.processor.
33                 get_decoder_prompt_ids(
34                     language=whisper_lang, task="transcribe"
35         )
36         else:
```

```

36         forced_ids = None # libre pour l'ewe (modele
           fine-tune)
37
38     # 4. Genere la transcription
39     with torch.no_grad():
40         predicted_ids = self.model.generate(
41             features,
42             forced_decoder_ids=forced_ids,
43             max_new_tokens=256,
44         )
45
46     # 5. Decode les tokens en texte
47     return self.processor.batch_decode(
48         predicted_ids, skip_special_tokens=True
49     )[0].strip()

```

Listing 5.1 – Service STT avec Whisper fine-tuné

5.5 Chargement du modèle : local ou Hugging Face Hub

Une particularité importante de notre implémentation est que la méthode `from_pretrained()` de la librairie `transformers` accepte indifféremment :

- Un **chemin local** (ex : `./crunch_modele_ewe`) si le dossier existe sur la machine.
- Un **identifiant Hugging Face Hub** (ex : `josiasetnm/crunch_modele_ewe`), auquel cas le modèle est téléchargé automatiquement.

Notre fichier `src/config.py` détecte automatiquement quel cas s'applique :

```

1 def _looks_like_local_path(value: str) -> bool:
2     if value.startswith(".") or value.startswith("/"):
3         return True
4     if os.path.isdir(value): # dossier existant sur le
       disque
5         return True
6     return False # sinon : identifiant Hugging Face Hub

```

Listing 5.2 – Détection automatique local vs Hub

Chapitre 6

Synthèse vocale (TTS)

6.1 Qu'est-ce que la synthèse vocale ?

Définition — TTS (Text-to-Speech)

La synthèse vocale, ou *Text-to-Speech* (TTS), est la tâche inverse de la reconnaissance vocale : elle convertit du texte écrit en signal audio (parole artificielle). C'est la technologie utilisée par les assistants vocaux, les livres audio automatiques, ou les GPS.

6.2 Architecture TTS adoptée

Nous avons choisi une architecture **hybride** pour couvrir nos trois langues avec la meilleure qualité possible tout en minimisant la charge mémoire :

Langue	Moteur TTS	Raison
Français	gTTS (Google Text-to-Speech)	API cloud, voix naturelle, légère
Anglais	gTTS (Google Text-to-Speech)	Idem
Ewe	Coqui TTS — VITS (OpenBible)	Seul modèle natif ewe disponible

6.2.1 gTTS pour le français et l'anglais

gTTS (*Google Text-to-Speech*) est une librairie Python qui appelle l'API Google Translate pour générer un fichier audio MP3 à partir d'un texte. Elle est légère, ne requiert aucun modèle local, et produit des voix de haute qualité.

```
1 from gtts import gTTS
2 import tempfile
3
```

```

4 tts = gTTS(text="Bonjour tout le monde", lang="fr")
5 tmp = tempfile.NamedTemporaryFile(delete=False, suffix=".mp3"
6     )
7 tts.save(tmp.name) # genere le fichier MP3

```

Listing 6.1 – Génération audio avec gTTS

6.2.2 Coqui TTS VITS pour l’ewe

Coqui TTS est un framework open source de synthèse vocale. Le modèle `tts_models/ewe/openbible/vits` est basé sur l’architecture **VITS** (*Variational Inference with adversarial learning for end-to-end Text-to-Speech*).

VITS est un modèle “bout-en-bout” (*end-to-end*) : il prend le texte en entrée et produit directement le signal audio, sans étapes intermédiaires séparées (contrairement aux anciens systèmes qui séparaient la prédiction de mélodie et la synthèse du signal).

Ce modèle a été entraîné sur le dataset **OpenBible ewe** — des enregistrements de passages bibliques lus à voix haute en ewe. C’est actuellement le seul modèle TTS natif disponible pour cette langue.

```

1 from TTS.api import TTS
2 import tempfile
3
4 model = TTS("tts_models/ewe/openbible/vits")
5
6 tmp = tempfile.NamedTemporaryFile(delete=False, suffix=".wav"
7     )
8 tmp.close()
9 model.tts_to_file(text="Ndi nyuie", file_path=tmp.name)
10 # genere le fichier WAV en ewe

```

Listing 6.2 – Génération audio en ewe avec Coqui VITS

6.3 Implémentation du service TTS avec chargement paresseux

Pour éviter de charger le modèle Coqui (lourd) au démarrage de l’application alors que l’utilisateur ne s’en est peut-être pas encore servi, nous utilisons un **chargement paresseux** (*lazy loading*) :

```

1 _ewe_tts_model = None # pas encore charge

```

```
2
3 def _get_ewe_tts_model():
4     global _ewe_tts_model
5     if _ewe_tts_model is None:
6         from TTS.api import TTS
7         _ewe_tts_model = TTS("tts_models/ewe/openbible/vits")
8     return _ewe_tts_model
9     # le modele n'est charge qu'une seule fois, au premier
       appel
```

Listing 6.3 – Service TTS avec lazy loading

Chapitre 7

Architecture logicielle

7.1 Vue d'ensemble

Notre application suit une architecture en **couches** :

Couche	Rôle
Interface (UI)	app/streamlit_app.py
Pipeline	src/pipelines/multilingual_pipeline.py
Services	src/services/{translation, stt, tts}_service.py
Configuration	src/config.py + .env

7.2 Le pipeline multilingue

La classe `MultilingualTranslatorPipeline` orchestre les trois services :

```
1 class MultilingualTranslatorPipeline:
2
3     def translate_text(self, text, source_lang, target_lang)
4         -> dict:
5         """Traduction texte -> texte."""
6         result = self._get_translation_service().translate(
7             text, source_lang, target_lang
8         )
9         return asdict(result)
10
11     def translate_audio(self, audio_path, audio_language,
12                        target_lang) -> dict:
13         """Audio -> transcription -> traduction."""
14         # Etape 1 : reconnaissance vocale
```



```

14         stt_result = self._get_stt_service().transcribe(
15             audio_path, source_language=audio_language
16         )
17         # Etape 2 : traduction du texte transcrit
18         translated = self._get_translation_service().
19             translate(
20                 stt_result.text, audio_language, target_lang
21             )
22         return {
23             "transcribed_text": stt_result.text,
24             "translated_text": translated.translated_text,
25         }

```

Listing 7.1 – Pipeline d’orchestration

7.3 Gestion de la configuration

La configuration est centralisée dans `src/config.py` et chargée depuis un fichier `.env`. Cela permet de modifier le comportement de l’application (modèles utilisés, device CPU/GPU, chemins) sans toucher au code source.

```

1 STT_MODEL_PATH=./crunch_modele_ewe          # local
2 # ou : STT_MODEL_PATH=josiasetnm/crunch_modele_ewe # Hugging
   Face Hub
3 TRANSLATION_MODEL_NAME=facebook/nllb-200-distilled-600M
4 OUTPUT_DIR=./outputs
5 DEVICE=cpu

```

Listing 7.2 – Variables d’environnement (.env)

7.4 Structure complète du projet

```

1 Projet_traduction_langue2/
2 |-- crunch_modele_ewe/          # Modele STT Whisper fine-
   tune (local)
3 | |-- config.json
4 | |-- model.safetensors
5 | |-- tokenizer.json
6 | |-- preprocessor_config.json
7 |-- src/
8 | |-- config.py                # Configuration centralisee
9 | |-- services/

```

```
10 | | |-- translation_service.py
11 | | |-- stt_service.py
12 | | |-- tts_service.py
13 | |-- pipelines/
14 |     |-- multilingual_pipeline.py
15 |-- app/
16 | |-- streamlit_app.py          # Interface Streamlit
17 |-- scripts/
18 | |-- prepare_parallel_data.py
19 | |-- train_translation_nllb.py
20 | |-- evaluate_translation.py
21 | |-- run_text_translate.py
22 | |-- run_audio_translate.py
23 |-- data/
24 | |-- raw/                      # Donnees brutes
25 | |-- processed/               # Donnees nettoyees
26 |-- Dockerfile                 # Deploiement Docker
27 |-- requirements.txt
28 |-- .env                       # Configuration locale (ne
    pas versionner)
29 |-- README.md
```

Listing 7.3 – Arborescence du projet

Chapitre 8

Interface utilisateur

8.1 Choix de Streamlit

Définition — Streamlit

Streamlit est une librairie Python qui permet de créer des applications web interactives en écrivant uniquement du Python, sans avoir besoin de connaître HTML, CSS ou JavaScript. C’est l’outil de prédilection pour les démos rapides en data science et en IA.

Nous avons choisi Streamlit pour sa rapidité de développement et sa parfaite intégration avec les librairies Python de machine learning (Hugging Face, PyTorch, etc.).

8.2 Organisation de l’interface

L’interface est organisée en **deux onglets** :

8.2.1 Onglet “Texte”

- Sélection des langues source et cible (Français, Anglais, Ewe).
- Bouton $\leftrightarrow$ pour inverser les langues et le texte.
- Zone de saisie du texte source.
- Zone de résultat (lecture seule) pour la traduction.
- Bouton **[Audio] Écouter** sous chaque zone (source et résultat) pour entendre la prononciation.

8.2.2 Onglet “Voix”

- Sélection des langues (langue parlée et langue cible de traduction).

- Enregistrement direct depuis le microphone (`st.audio_input`).
- Import d'un fichier audio (.wav, .mp3, .m4a).
- Bouton **Transcrire et traduire** qui déclenche le pipeline complet.
- Affichage de la transcription et de la traduction.
- Bouton **[Audio] Écouter** sur chacun des résultats.

8.3 Optimisation : cache des modèles

Les modèles (NLLB, Whisper, Coqui) sont lourds à charger (plusieurs centaines de mégaoctets, plusieurs dizaines de secondes). Pour éviter de les recharger à chaque interaction utilisateur, nous utilisons le décorateur `@st.cache_resource` :

```
1 @st.cache_resource(show_spinner=False)
2 def load_pipeline() -> MultilingualTranslatorPipeline:
3     return MultilingualTranslatorPipeline()
4 # Les modeles ne sont charges qu'une seule fois,
5 # quelle que soit la frequence des interactions.
```

Listing 8.1 – Cache Streamlit des modèles

Chapitre 9

Déploiement en ligne

9.1 Hugging Face Hub et Hugging Face Spaces

Définition — Hugging Face Hub

Le Hugging Face Hub est une plateforme en ligne permettant d'héberger, de partager et de télécharger des modèles de machine learning, des datasets et des applications. C'est le GitHub de l'IA.

Définition — Hugging Face Spaces

Hugging Face Spaces est un service d'hébergement d'applications de machine learning, directement intégré au Hub. Il permet de déployer une application Streamlit, Gradio ou Docker en quelques commandes, avec une URL publique accessible à tous.

9.2 Hébergement du modèle STT

Le modèle `crunch_modele_ewe` (plusieurs centaines de mégaoctets) ne peut pas être versionné dans Git (limite de 100 Mo par fichier sur GitHub/Hugging Face). Il est hébergé séparément sur le Hugging Face Hub :

```
1 # Creation du repo modele
2 hf repo create crunch_modele_ewe --repo-type model
3
4 # Upload de tous les fichiers du dossier
5 huggingface-cli upload josiasetnm/crunch_modele_ewe \
6     ./crunch_modele_ewe .
```

Listing 9.1 – Upload du modèle sur Hugging Face Hub

Le modèle est accessible publiquement à l'adresse : https://huggingface.co/josiasetnm/crunch_modele_ewe

9.3 Containerisation avec Docker

Définition — Docker et conteneur

Docker est un outil qui permet d'empaqueter une application et toutes ses dépendances dans un **conteneur** — une sorte de “boîte” autonome qui fonctionne de façon identique sur n'importe quel serveur, indépendamment de l'environnement hôte.

Notre application est containerisée via un Dockerfile :

```
1 FROM python:3.11-slim
2
3 WORKDIR /app
4
5 # Dependances systeme pour torchaudio, librosa, Coqui TTS
6 RUN apt-get update && apt-get install -y --no-install-
    recommends \
7     ffmpeg libsndfile1 git \
8     && rm -rf /var/lib/apt/lists/*
9
10 COPY requirements.txt .
11 RUN pip install --no-cache-dir -r requirements.txt
12
13 COPY . .
14
15 ENV PYTHONPATH=/app
16 EXPOSE 8501
17
18 CMD ["streamlit", "run", "app/streamlit_app.py",
19     "--server.port=8501", "--server.address=0.0.0.0",
20     "--server.fileWatcherType=none"]
```

Listing 9.2 – Dockerfile de l'application

Explication :

- FROM python:3.11-slim : image de base Linux avec Python 3.11, version allégée (*slim*) pour réduire la taille du conteneur.
- ffmpeg et libsndfile1 : bibliothèques système nécessaires pour le traitement audio (librosa, torchaudio, Coqui).

- `-server.fileWatcherType=none` : désactive le watcher de fichiers de Streamlit en production (inutile en ligne, génère du bruit dans les logs).

9.4 Déploiement sur Hugging Face Spaces

```
1 # 1. Creation du Space avec SDK Docker
2 hf repo create traducteur-multilingue-ewe \
3     --repo-type space --space_sdk docker
4
5 # 2. Clonage du Space vide
6 git clone https://huggingface.co/spaces/josiasetnm/\
7 traducteur-multilingue-ewe
8 cd traducteur-multilingue-ewe
9
10 # 3. Copie du code (sans .venv, crunch_modele_ewe, .env)
11 # ... copier manuellement ou avec rsync ...
12
13 # 4. Premier commit et push
14 git add .
15 git commit -m "Deploiement initial"
16 git push
```

Listing 9.3 – Déploiement complet étape par étape

Les variables d'environnement sont configurées dans l'interface web du Space (Settings → Variables and secrets) :

```
1 STT_MODEL_PATH          = josiasetnm/crunch_modele_ewe
2 TRANSLATION_MODEL_NAME  = facebook/nllb-200-distilled-600M
3 DEVICE                  = cpu
```

Listing 9.4 – Variables d'environnement du Space

L'application est désormais accessible publiquement à l'adresse : <https://huggingface.co/spaces/josiasetnm/traducteur-multilingue-ewe>

Chapitre 10

Librairies utilisées

Librairie	Usage	Description
torch (PyTorch)	Base deep learning	Framework de calcul tensoriel et d'entraînement de réseaux de neurones développé par Meta.
torchaudio	Traitement audio	Extension PyTorch pour le chargement et la transformation de signaux audio.
transformers	Modèles NLP	Librairie Hugging Face donnant accès à des milliers de modèles pré-entraînés (NLLB, Whisper, BERT, GPT, etc.).
librosa	Analyse audio	Librairie Python pour l'analyse et le traitement du signal audio. Utilisée pour charger les fichiers audio et les rééchantillonner à 16 kHz.
gtts	TTS fr/en	Google Text-to-Speech. Génère des fichiers MP3 via l'API Google.
coqui-tts	TTS ewe	Framework open source de synthèse vocale avec modèle natif ewe (VITS OpenBible).
streamlit	Interface web	Création d'applications web interactives en Python pur.
huggingface_hub	Hub client	Client Python pour interagir avec le Hugging Face Hub (upload, download de modèles).

Librairie	Usage	Description
python-dotenv	Configuration	Chargement de fichiers <code>.env</code> pour gérer les variables d'environnement.
datasets	Gestion données	Librairie Hugging Face pour charger et manipuler des datasets.
evaluate + sacre-bleu	Évaluation	Calcul des métriques BLEU et ROUGE pour évaluer la qualité des traductions.
sentencepiece	Tokenisation	Librairie de tokenisation en sous-mots, requise par NLLB.
pandas	Manipulation CSV	Manipulation de DataFrames pour le prétraitement des corpus.
numpy	Calcul numérique	Calcul matriciel bas niveau, utilisé par presque toutes les autres librairies.

Chapitre 11

Évaluation et résultats

11.1 Métriques d'évaluation de la traduction

11.1.1 Score BLEU

Définition — Score BLEU

Le BLEU (*Bilingual Evaluation Understudy*) est la métrique la plus utilisée pour évaluer la qualité de la traduction automatique. Il mesure le chevauchement entre la traduction produite par le modèle et une traduction de référence faite par un humain.

$$\text{BLEU} = \text{BP} \times \exp\left(\sum_{n=1}^N w_n \log p_n\right)$$

où p_n est la précision des n-grammes (séquences de n mots consécutifs) et BP est un facteur de pénalité pour les traductions trop courtes.

Le score BLEU varie de 0 à 100 : 0 signifie aucune correspondance, 100 signifie une correspondance parfaite avec la référence. En pratique, un score de 30–40 est considéré comme correct pour un système de traduction automatique.

11.1.2 Commande d'évaluation

```
1 python scripts/evaluate_translation.py \  
2 --csv data/processed/test_fr_ewe.csv \  
3 --source-col fr_text \  
4 --target-col ewe_text \  
5 --source-lang fr \  
6 --target-lang ewe
```

Listing 11.1 – Évaluation BLEU

11.2 Tests de l’interface

L’application a été testée pour les fonctionnalités suivantes :

Fonctionnalité	Résultat	Observations
Traduction fr → en	✓ Fonctionnel	Qualité élevée
Traduction fr → ewe	✓ Fonctionnel	Qualité correcte
Traduction ewe → fr	✓ Fonctionnel	Qualité correcte
Bouton swap $\Leftrightarrow$ langues	✓ Fonctionnel	Inverse texte et langues
TTS français (gTTS)	✓ Fonctionnel	Voix naturelle
TTS anglais (gTTS)	✓ Fonctionnel	Voix naturelle
TTS ewe (Coqui VITS)	✓ Fonctionnel	Voix native ewe
STT ewe (micro/fichier)	✓ Fonctionnel	Modèle fine-tuné
Pipeline audio complet	✓ Fonctionnel	Audio → texte → traduction
Déploiement en ligne	✓ Accessible	HF Spaces

Chapitre 12

Limites et perspectives

12.1 Limites connues

12.1.1 Volume de données ewe

Le corpus ewe disponible est limité (principalement le corpus JW300, issu de textes religieux). Cela a deux conséquences :

- La qualité des traductions impliquant l’ewe est inférieure à celle des traductions fr $\leftrightarrow$ en, pour lesquelles NLLB a été entraîné sur des milliards de phrases.
- Le modèle peut peiner sur des domaines spécifiques (médical, juridique, technique) peu représentés dans les données d’entraînement.

12.1.2 TTS ewe — corpus religieux

Le modèle Coqui VITS ewe est entraîné uniquement sur des lectures bibliques. La prosodie (rythme, intonation) peut donc différer de l’ewe conversationnel courant.

12.1.3 Performances en CPU

L’ensemble des modèles fonctionne sur CPU (pas de GPU requis), ce qui le rend accessible sur des machines standard. En contrepartie, les temps de réponse sont de l’ordre de 1 à 5 secondes par traduction, et le premier chargement des modèles prend 30 secondes à 2 minutes.

12.2 Perspectives d’amélioration

1. **Enrichissement du corpus ewe** : collaboration avec des locuteurs natifs pour créer un dataset annoté dans des domaines variés (santé, éducation, quotidien).

2. **Fine-tuning de NLLB sur le corpus spécifique** : utiliser les scripts `train_translation_nllb` pour affiner NLLB sur nos données ewe collectées.
3. **Amélioration du TTS ewe** : entraîner un modèle VITS sur un corpus conversationnel ewe (non religieux) pour une voix plus naturelle.
4. **Augmentation des données par back-translation** : générer automatiquement des paires fr-ewe en traduisant des textes français en ewe avec le modèle actuel, puis retraduire en français pour filtrer les meilleures traductions.
5. **Accélération avec GPU** : déployer sur un Space Hugging Face avec accélérateur GPU pour réduire les temps de réponse à moins d’une seconde.
6. **Application mobile** : portage vers Flutter ou React Native pour une utilisation sur smartphone, particulièrement adapté aux contextes africains où le mobile est dominant.

12.3 Conclusion

Ce projet démontre la faisabilité d’un système de traduction automatique et vocale pour l’ewe, une langue africaine à faibles ressources, en exploitant des modèles pré-entraînés de pointe (NLLB-200, Whisper, VITS) et le paradigme du transfer learning.

L’architecture mise en place — services découplés, configuration externalisée, interface Streamlit, déploiement Docker sur Hugging Face Spaces — constitue une base solide et extensible pour de futurs travaux. Le système est aujourd’hui accessible à n’importe qui via le lien :

<https://huggingface.co/spaces/josiasetnm/traducteur-multilingue-ewe>

Ce projet contribue modestement, mais concrètement, à la réduction de la fracture numérique linguistique pour les locuteurs ewe, et illustre comment des équipes académiques à ressources limitées peuvent tirer parti des avancées récentes en IA pour des applications à fort impact social.

Annexe A

Glossaire

API *Application Programming Interface*. Interface permettant à deux programmes de communiquer entre eux.

BLEU *Bilingual Evaluation Understudy*. Métrique standard d'évaluation de la traduction automatique.

BPE *Byte Pair Encoding*. Algorithme de tokenisation en sous-mots.

Corpus Ensemble de textes ou d'enregistrements audio utilisés pour entraîner ou évaluer un modèle.

Deep learning Apprentissage profond. Branche du machine learning utilisant des réseaux de neurones à plusieurs couches.

Docker Outil de containerisation permettant d'empaqueter une application et ses dépendances.

Embedding Représentation vectorielle (numérique) d'un mot ou d'un token.

Fine-tuning Ré-entraînement partiel d'un modèle pré-entraîné sur une tâche spécifique.

GPU *Graphics Processing Unit*. Processeur graphique, bien adapté aux calculs matriciels du deep learning.

Hugging Face Entreprise et plateforme open source hébergeant des modèles et datasets de machine learning.

Inférence Utilisation d'un modèle entraîné pour faire des prédictions sur de nouvelles données (par opposition à l'entraînement).

Low-resource Se dit d'une langue pour laquelle peu de données numériques sont disponibles.

NLP *Natural Language Processing*. Traitement automatique du langage naturel.

NLLB *No Language Left Behind*. Modèle de traduction de Meta supportant 200 langues.

- Paramètre** Valeur numérique apprise par un réseau de neurones pendant l'entraînement.
- Pré-entraînement** Entraînement initial d'un modèle sur un large corpus générique.
- STT** *Speech-to-Text*. Reconnaissance vocale automatique.
- Token** Unité de base du texte pour un modèle NLP (mot, sous-mot ou caractère).
- Transfer learning** Réutilisation des connaissances d'un modèle pré-entraîné pour une nouvelle tâche.
- Transformer** Architecture de réseau de neurones basée sur le mécanisme d'attention, base des modèles NLP modernes.
- TTS** *Text-to-Speech*. Synthèse vocale automatique.
- VITS** Architecture de TTS bout-en-bout basée sur l'inférence variationnelle.
- WER** *Word Error Rate*. Taux d'erreur par mot, métrique d'évaluation de la reconnaissance vocale.
- Whisper** Modèle de reconnaissance vocale d'OpenAI, entraîné sur 680 000 heures d'audio multilingue.